BigQuery AI.GENERATE関数の活用

SQLだけで始める生成AI

データ分析における生成AIの課題

従来の課題

- **外部API呼び出し**: 認証・エラーハンドリング・レート制限の管理が複雑
- **データガバナンス**: 機密データの外部送信リスク
- **SQL統合**: ETLパイプラインが複雑化

AI.GENERATE関数で解決

SQLネイティブ で **セキュア** な生成AI活用が可能に

2

AI.GENERATEの3つの強み

1. Multimodal （マルチモーダル）

テキスト・画像・動画・音声・PDFを統一インターフェースで処理

2. SQL-Native （SQL統合）

SELECT・WHERE・JOINで直接使用、既存クエリに簡単統合

3. Structured Output （構造化出力）

JSONスキーマ指定で型安全な構造化データを生成

3

GENERATE関数の種類と違い（混同注意！）

関数名	処理単位	入力対応	特徴
AI.GENERATE	Row-wise	マルチモーダル	**推奨・汎用的**
AI.GENERATE_TEXT	Table-valued	テキスト	従来型
AI.GENERATE_TABLE	Row-wise	テキスト	構造化データ
ML.GENERATE_TEXT	Table-valued	テキスト	旧式
GENERATE_DATE_ARRAY	-	日付	**別物（配列生成）**

ポイント: AI.GENERATEが最も汎用的で使いやすい

4

最小構成で始める

基本構文

```
SELECT AI.GENERATE(
  'Summarize this text',                    -- 1. プロンプト
  'Your content here',                      -- 2. 入力データ
  STRUCT('gemini-2.5-flash' AS model)      -- 3. モデル指定
) AS result
```

セットアップ

- ✓ **Vertex AI API** の有効化のみ
- ✓ モデルデプロイ **不要**
- ✓ 追加設定 **不要**

5

実践例1: テキスト要約

```sql
-- カスタマーレビューを50単語で要約
SELECT
  review_id,
  review_text,
  AI.GENERATE(
    'Summarize in 50 words',
    review_text,
    STRUCT('gemini-2.5-flash' AS model)
  ) AS summary
FROM
  customer_reviews
WHERE
  rating <= 2  -- 低評価レビューのみ
LIMIT 100;
```

実行結果: 100件のレビューを一括要約（約30秒）

6

実践例2: 構造化出力

```sql
-- ユーザープロフィールから情報抽出
SELECT
  user_id,
  AI.GENERATE(
    'Extract name, age, occupation',
    user_bio,
    STRUCT(
      'name STRING, age INT64' AS output_schema
    )
  ) AS data
FROM user_profiles;
```

ポイント: `output_schema` でJSONスキーマ指定

7

実践例3: 画像分析

```sql
-- 商品画像から説明文を自動生成
SELECT
  product_id,
  AI.GENERATE(
    'Describe this image',
    OBJ.GET_ACCESS_URL(image_uri),
    STRUCT('gemini-2.5-flash' AS model)
  ) AS description
FROM product_images
WHERE description IS NULL;
```

ポイント: `OBJ.GET_ACCESS_URL` でObject Table参照

用途: 画像、動画、音声、PDFすべて同じ構文

8

ベストプラクティス

💰 コスト最適化

- **Batch API**: 自動適用（Gemini 2.0+）
- **モデル選択**: Flash（高速）/ Pro（高精度）
- **事前フィルタ**: WHERE句で処理量削減

⚡ パフォーマンス

- **大規模処理**: 100万〜1000万行 / 6時間
- **並列実行**: BigQueryの分散処理を活用
- **キャッシュ**: 結果の再利用でコスト削減

9

他手法との比較

手法	SQL統合	セットアップ	総合評価
AI.GENERATE	◎	最小限	**◎推奨**
Cloud Functions	△	複雑	○
Vertex AI直接	×	中程度	○
外部API	×	複雑	△

選択ガイド

- 📊 **SQLで完結** したい → **AI.GENERATE**
- 🔧 **高度なカスタマイズ** → Cloud Functions
- 🚀 **最新モデル優先** → Vertex AI直接

10

まとめ

メリット

- **SQLだけで完結** - 学習コスト最小
- **データ移動不要** - セキュリティ◎
- **2026年GA化** - Gemini 3.0対応

制限事項

- Vertex AIへの依存
- コスト管理が必要
- プロンプト設計の工夫

参考リンク

公式ドキュメント

- AI.GENERATE function
- Generative AI overview
- Choose a text generation function

チュートリアル

- Generate text tutorial (Gemini)
- Generate structured data

ブログ記事

- New BigQuery gen AI functions
- SQL reimagined for the AI era
- NRIネットコム: AI.GENERATEを試してみた

12